Python e numpy per l'Assignment 4

Guida passo passo, cresce con il progetto

Filippo Ferroni

Versione 1.2, 25 ottobre 2026 — Lezioni 1–2: funzioni, algebra lineare, dati casuali

Come usare questa guida. Ogni lezione corrisponde a un pezzo del progetto. Si impara solo il Python che serve, nel momento in cui serve. Alla fine di ogni lezione ci sono esercizi; le soluzioni dei TODO del progetto vengono aggiunte *dopo* che li hai scritti tu.

Indice

1 La mappa del progetto **1**

2 Eseguire un file Python **1**

3 Variabili e `print` **1**

4 Funzioni **2**

5 numpy: vettori e matrici **2**

6 Operazioni: elemento per elemento vs prodotto matriciale **2**

7 Dalla formula al codice: il metodo in 4 passi **3**
7.1 Esempio svolto: la ridge (TODO 2) 3

8 Errori frequenti e cosa vogliono dire **4**

9 Esercizi della lezione 1 **4**

10 Lezione 2: generare dati (`src/data.py`) **5**
10.1 Numeri casuali riproducibili: il seed 5
10.2 Normalizzare un vettore 5
10.3 Slicing: prendere un pezzo di matrice 5
10.4 Restituire più valori 5
10.5 Perché il check C dà circa 0,25 5

1 La mappa del progetto

File	Cosa fa	Lezione
`src/models.py`	least squares, ridge, errore quadratico medio	1 (questa)
`src/data.py`	genera dati sintetici, carica il dataset reale, split train/test	2
`src/experiment.py`	ciclo su d: addestra e misura gli errori	3
`src/plots.py`	grafici con matplotlib	4

2 Eseguire un file Python

- Un file `.py` è testo con istruzioni eseguite dall'alto verso il basso.
- Dal terminale, dentro la cartella del repo:

```
python src/models.py
```

  (su Mac a volte il comando è `python3`).

- In VS Code: apri la cartella `smml-double-descent`, apri il file, poi usa il pulsante "Run" in alto a destra.
- Le righe che iniziano con `#` sono **commenti**: Python le ignora, servono a chi legge.

3 Variabili e `print`

```
m = 3              # un numero intero
alpha = 0.1        # un numero decimale (si usa il punto, non la virgola)
nome = "ridge"     # una stringa (testo)
print("alpha vale", alpha)
```

Il segno `=` **assegna** un valore a un nome; non è un'uguaglianza matematica.

4 Funzioni

Una funzione è una ricetta: riceve degli input, fa dei calcoli, restituisce un risultato.

```
def quadrato(x):
    """Restituisce x al quadrato."""   # docstring: spiega cosa fa
    risultato = x ** 2                 # ** e' la potenza
    return risultato

print(quadrato(4))   # stampa 16
```

- `def` definisce la funzione; tra parentesi gli **argomenti** (input).
- Il corpo è **rientrato di 4 spazi**. Il rientro dice a Python quali righe appartengono alla funzione. Se è sbagliato: `IndentationError`.
- `return` restituisce il risultato. Senza `return`, la funzione restituisce `None` ("niente").
- `raise NotImplementedError` vuol dire "non ancora scritto": lo trovi nei TODO, va sostituito con il tuo codice.

Il blocco `if __name__ == "__main__":` Il codice sotto questa riga viene eseguito solo quando lanci il file direttamente (`python src/models.py`), non quando un altro file fa `import` di `models.py`. Lo usiamo per i controlli rapidi.

5 numpy: vettori e matrici

```python
import numpy as np        # importa numpy con il nome breve np

y = np.array([1.0, 2.0, 3.0])            # vettore con 3 elementi
X = np.array([[1.0, 0.0],
              [1.0, 1.0],
              [1.0, 2.0]])               # matrice 3 x 2 (3 righe, 2 colonne)

print(X.shape)       # (3, 2)
print(X.shape[0])    # 3 = numero di righe = m
print(X.shape[1])    # 2 = numero di colonne = d
print(X[0])          # prima riga: [1. 0.]   (si conta da 0!)
print(X[2, 1])       # riga 3, colonna 2: 2.0
```

Attenzione: Python conta da 0. Il primo elemento è `[0]`, non `[1]`.

6 Operazioni: elemento per elemento vs prodotto matriciale

Matematica	numpy	Note
$A + B$	`A + B`	elemento per elemento, stesse dimensioni
αA	`alpha * A`	moltiplica ogni elemento per α
v_i^2 per ogni i	`v ** 2`	potenza elemento per elemento
AB (prodotto matriciale)	`A @ B`	**non** `A * B`!
$A^\top$	`A.T`	trasposta
I_d	`np.eye(d)`	identità $d \times d$
$\frac{1}{m}\sum_i v_i$	`np.mean(v)`	media degli elementi
$\|w\|$	`np.linalg.norm(w)`	norma euclidea
A^+	`np.linalg.pinv(A)`	pseudo-inversa
risolvi $Aw = b$	`np.linalg.solve(A, b)`	meglio che calcolare A^{-1}

Errore classico: `A * B` moltiplica elemento per elemento, *non* è il prodotto righe per colonne. Per le formule delle slide serve quasi sempre `@`.

La regola delle dimensioni. `A @ B` funziona solo se il numero di colonne di `A` è uguale al numero di righe di `B`: $(p \times q) \,@\, (q \times r) = (p \times r)$. Se non torna, numpy dà un errore che contiene la parola `mismatch`. **Prima di scrivere una riga, controlla le dimensioni a mano.**

7 Dalla formula al codice: il metodo in 4 passi

1. Scrivi la formula matematica.
2. Dai un nome a ogni pezzo (es. A, b).
3. Traduci ogni simbolo con la tabella della sezione precedente.
4. Restituisci il risultato con `return`.

7.1 Esempio svolto: la ridge (TODO 2)

Formula (slide 9, p. 5): $(\alpha I + X^\top X)\, w = X^\top y$, cioè $Aw = b$ con $A = \alpha I + X^\top X$, $b = X^\top y$.

```python
def fit_ridge(X, y, alpha):
    d = X.shape[1]                      # numero di feature (colonne di X)
    A = alpha * np.eye(d) + X.T @ X     # matrice d x d
```

```
    b = X.T @ y                          # vettore con d elementi
    w = np.linalg.solve(A, b)            # risolve A w = b
    return w
```

Controllo delle dimensioni con X di forma $m \times d$ e y con m elementi:

- `np.eye(d)`: $d \times d$; `X.T @ X`: $(d \times m)(m \times d) = d \times d$; somma: $d \times d$. ✓
- `X.T @ y`: $(d \times m)(m) = d$. ✓
- `solve`: da una matrice $d \times d$ e un vettore d restituisce w con d elementi. ✓

8 Errori frequenti e cosa vogliono dire

Messaggio	Causa probabile
`IndentationError`	rientro sbagliato (usa 4 spazi, non mescolare tab e spazi)
`NameError: name 'np' is not defined`	manca `import numpy as np` in cima al file
`NotImplementedError`	hai lasciato la riga `raise NotImplementedError` nel TODO
`ValueError: matmul: ... mismatch`	dimensioni incompatibili in un `@`: controlla le forme
`LinAlgError: Singular matrix`	`solve` con una matrice non invertibile (es. ridge con $\alpha = 0$ e $d > m$)
`ModuleNotFoundError: numpy`	numpy non installato: `pip install -r requirements.txt`

9 Esercizi della lezione 1

1. Con X di forma 3×2: perché `X @ X` dà errore, mentre `X.T @ X` funziona? Che forma ha il risultato?
2. Che differenza c'è tra `X.T @ X` e `X @ X.T`? Che forme hanno con $m = 3$, $d = 2$? (Indizio: una delle due compare nella formula della minima norma, teoria sez. 4.3.)
3. Che cosa stampa `np.array([1.0, -2.0, 3.0]) ** 2`? E `np.mean` di quel risultato?
4. **TODO 1** di `models.py`: traduci $w = X^{+}y$ con il metodo in 4 passi.
5. **TODO 3** di `models.py`: traduci $\frac{1}{m}\|Xw - y\|^2$. (Indizio: prima il vettore degli errori, poi la media dei quadrati.)

Soluzioni (esercizi 1–3)

1. `X @ X` richiede colonne di X (2) = righe di X (3): falso, quindi errore. `X.T @ X` è $(2 \times 3)(3 \times 2) = 2 \times 2$.
2. `X.T @ X` è $d \times d = 2 \times 2$ (equazioni normali, ridge); `X @ X.T` è $m \times m = 3 \times 3$ (compare in $w^{+} = X^{\top}(XX^{\top})^{-1}y$).
3. `[1. 4. 9.]`; la media è $14/3 \approx 4.67$.

Soluzioni dei TODO (scritte da te, corrette il 25/10)

```python
def fit_least_squares(X, y):
    w = np.linalg.pinv(X) @ y          # w = X^+ y, minima norma per ogni d
    return w

def mse(X, y, w):
    y_hat = X @ w                      # predizioni sul set (m valori)
    error = np.mean((y_hat - y) ** 2)  # media dei quadrati degli errori
    return error
```

Output dei controlli: A $= [1, 1]$, B $= [1, 10]$, C $\approx [1.0, 0.01]$, D $\approx 2 \cdot 10^{-32}$ (zero numerico: il computer lavora con circa 16 cifre significative, quindi un valore così piccolo è uno zero). **Lezione dal check C:** sugli stessi dati la minima norma mette un peso pari a 10, la ridge 0.01: è il meccanismo che elimina il picco, visto sul tuo codice.

10 Lezione 2: generare dati (`src/data.py`)

A cosa serve. Per vedere il double descent servono dati di cui conosciamo la "verità": $y = w^{*\top} x +$ rumore, con $x \in \mathbb{R}^D$ ($D = 500$ feature "deboli"), $m = 100$ esempi di training e 2000 di test. Il modello userà solo le prime d feature: facendo crescere d da 1 a 500 attraversiamo la soglia $d = m$.

10.1 Numeri casuali riproducibili: il seed

```python
rng = np.random.default_rng(42)    # generatore con seed 42
a = rng.normal(size=3)             # 3 numeri da N(0,1)
```

- `rng` è un oggetto "generatore". Lo stesso seed produce **sempre** la stessa sequenza: esperimenti riproducibili (requisito dell'assignment).
- `rng.normal(size=...)` estrae da una normale standard $N(0, 1)$. `size` decide la forma: `size=5` dà un vettore di 5 valori, `size=(100, 500)` una matrice 100×500 (una **tupla** tra parentesi tonde).
- Rumore con deviazione standard σ: `sigma * rng.normal(size=m)`, perché se $Z \sim N(0, 1)$ allora $\sigma Z \sim N(0, \sigma^2)$.

10.2 Normalizzare un vettore

`w_star / np.linalg.norm(w_star)` divide ogni componente per la norma: il risultato ha norma 1. Così il segnale $w^{*\top} x$ ha varianza 1 e il rapporto segnale/rumore è controllato solo da σ.

10.3 Slicing: prendere un pezzo di matrice

Codice	Significato
`X[:, :d]`	tutte le righe, colonne da 0 a $d-1$ (le prime d feature)
`X[:10]`	le prime 10 righe
`X[:, 0]`	la prima colonna (vettore)
`X[5:, 2:4]`	righe dalla 5 in poi, colonne 2 e 3

Regola: `a:b` include `a` ed **esclude** `b`; `:` da solo significa "tutto".

10.4 Restituire più valori

```
return X_train, y_train, X_test, y_test, w_star        # nella funzione
X_tr, y_tr, X_te, y_te, w = make_synthetic(100, 2000, 500, 0.5, 0)   # chi la chiama
```

Python impacchetta i valori in una tupla e li "spacchetta" nell'ordine in cui li hai scritti.

10.5 Perché il check C dà circa 0,25

Con $\sigma = 0{,}5$ anche il vero w^* sbaglia di $\sigma^2 = 0{,}25$ in media: è il **rischio di Bayes** per la square loss (statRisk p. 1–2). Nessun modello può fare meglio sul test: nei grafici sarà la linea orizzontale di riferimento. Il valore esatto (0,256) oscilla un po' perché il test ha un numero finito di esempi.

Esercizi lezione 2

1. Che forma ha `rng.normal(size=(3, 4))`? E `X[:, :2]` se X è 3×4?
2. Perché generiamo il test set con lo *stesso* `rng` ma *dopo* il training? Cosa garantisce?
3. Se metti `noise_std=1.0`, a quanto deve tendere il check C?

Soluzioni

1. $(3, 4)$; $(3, 2)$.
2. Le estrazioni sono indipendenti: nessun esempio è condiviso tra train e test (niente data leakage), e il seed rende tutto riproducibile.
3. A 1 ($= \sigma^2$).

Soluzioni dei TODO (scritte da te, verificate il 25/10)

```
X_train = rng.normal(size=(m, D))                                  # TODO 1
y_train = X_train @ w_star + noise_std * rng.normal(size=m)  # TODO 2
return X[:, :d]                                                    # TODO 3
```

Output: A forme corrette, B $(100, 30)$, C 0,2564, D `True`. **Pulizia:** la riga `raise NotImplementedError` rimasta dopo il `return` non viene mai eseguita (codice morto): cancellala.